Machine Learning for Cross-Platform User Access Reviews

AccessGuard project proposal

Prepared for Hashan Hirantha | Revised 3 November 2026

Executive summary

We propose a machine learning prototype that reads user account inventories and activity logs, then identifies accounts that need human attention during periodic access reviews. The prototype will support Linux accounts, Windows accounts and application inventories supplied as CSV files. It will detect unusual inactivity and changes in an account's observed behavior, and produce a ranked review report with supporting evidence.

The main deliverable is a trained anomaly-detection model with a reproducible Python pipeline and a command-line report. We will evaluate Isolation Forest against simple review rules and a statistical change baseline. A small Linux and Windows lab, with an authenticated Apache application where feasible, will provide actual login and activity events. Account inventories may include mock records. Production access to Dialog systems and a web interface are not prerequisites.

Problem and operational context

Organizations must periodically check whether accounts remain necessary and whether their privileges still match their owners' responsibilities. The stakeholder discussion describes quarterly reviews and a workload that becomes difficult when administrators must inspect many accounts individually [1]. A central directory can supply useful information, but local operating-system accounts and application users may exist outside that directory. A portable solution must therefore accept multiple inventory and log formats.

The required output is a decision aid: which accounts warrant review now, and which show no current review trigger within the evidence available. An account that changes from network-administration activity to database-related activity illustrates a review case; the change may be legitimate, but its access should be checked. Login logs alone cannot establish such a change unless relevant activity is also recorded.

Aim and objectives

The aim is to reduce routine review effort while making exceptional accounts and evidence gaps visible. We will:

- Normalize Linux, Windows and CSV account inventories together with attributable activity events.
- Build account-level features that describe activity, inactivity and change against an earlier baseline.
- Train an anomaly model and generate evidence-backed review recommendations.
- Evaluate review-case detection, false alerts, evidence coverage and processing cost on held-out lab scenarios.
- Deliver the trained artifact, source code, documented data and an executable demonstration.

Stakeholder requirements and project scope

The following scope follows the stakeholder discussion [1]. Specific algorithms, feature windows and the delivery schedule below are proposed implementation choices and remain subject to feasibility testing.

Requirement	Prototype commitment	Priority
Account inputs	Support Linux and Windows inventories plus a documented CSV schema for other systems.	Core
Behavior inputs	Process login, system and activity logs that can be attributed to an account.	Core
ML output	Identify unusual accounts and provide a ranked list for human access review.	Core
Data environment	Use sample operating systems or VMs. Obtain actual activity events from the lab.	Core
Portability	Keep source mappings configurable; avoid dependencies on Dialog-specific systems or data.	Core
Directory integration	Treat Microsoft AD or another IAM export as an additional source, not the only account authority.	Optional
Web interface	Use a command-line workflow and CSV/JSON reports first. Add a small viewer only if time remains.	Optional
Account changes	Recommend review without automatically disabling accounts or changing privileges.	Out of scope

Functional requirements

The prototype must validate account and event inputs, preserve their source and assessment time, create versioned feature snapshots, fit and reload the model, score every eligible account, and export a reproducible review report. Invalid or unattributable records must appear in a data-quality summary rather than silently disappearing. Each result must identify the source account, model version, evidence window and reasons for review.

Scope boundaries

The first release will analyze interactive human accounts in the lab. Built-in service accounts and emergency accounts will be explicitly identified and excluded from learned comparisons or handled by separate policies. CSV support extends the inventory format; behavior scoring still requires suitable linked logs. Full identity provisioning, enterprise connectors, approval orchestration and automated remediation are later extensions.

The existing AccessGuard web prototype may be retained as a demonstration shell. Its current fixed risk rules do not constitute the required trained model. Development effort will first move to inventory normalization, log processing, feature construction, model training and report generation.

Data collection and feature engineering

Lab and source collection

Begin with one Linux VM and one Windows VM. Add an authenticated Apache application to demonstrate application accounts supplied through CSV if resources permit. Create a small set of named lab roles and approximately 10-20 interactive test accounts across the hosts. This is a feasibility target, not a statistically sufficient enterprise sample. Start collecting baseline events immediately.

Source	Inventory and events	Collection condition
Linux	Account and group exports; SSH/login outcomes; attributable sudo or audited activity.	Use journal or authentication logs available on the chosen distribution. Enable additional auditing only for the activity required [2].
Windows	Local users and group memberships; successful/failed logons; selected activity events.	PowerShell supports inventory export [3]. Verify audit policy: 4624 records successful logon; 4688 process events require process-creation auditing [4,5].
CSV application	Account identifier, role and status; authenticated application-access events.	Apache can record the authenticated remote user. Anonymous requests cannot be assigned to a person using IP address alone [6].

Common data contract

Inventory fields will include system_id, source_account_id, account_name, platform, account_type, role/group, enabled_status, observed_at and optional last_active_at with provenance. Event fields will include system_id, source_account_id, event_time, event_type, outcome, activity_category and source_event_id. Keep each account keyed by system plus its source identifier; identical usernames on different machines are not automatically the same identity.

Features available at assessment time

- Activity: successful and failed logon counts, active periods, session frequency and event-category counts.
- Inactivity: time since a verified successful event, together with the amount and freshness of log coverage.
- Change: current activity frequency and category mix compared with that account's earlier observed baseline.
- Context and quality: administrative role, recent group changes, missing fields, source outages and history length.

Normalize timestamps to UTC while retaining the original timezone. Deduplicate events and record parsing failures. Authentication logs do not automatically reveal all commands or business activities. Features about database work or privilege use are enabled only when those events are actually observable. Never interpret missing logs as proof of inactivity.

Use lab identities and minimize retained fields. Passwords, password hashes, tokens and /etc/shadow contents are unnecessary inputs. Keep raw logs access-controlled and preserve their checksums and collection configuration.

Machine learning method and review output

Prediction unit and historical reference

One example is an account assessed over a defined activity window. Start with a previous seven-day reference and a current one-day window where enough data exists; expose both as configuration. Build historical training examples using only events available before their assessment time. Cold-start accounts receive an explicit insufficient-history result until their evidence meets the configured requirements.

Model training

Isolation Forest is the initial learned model. It fits an ensemble that isolates unusual feature combinations and can be trained without binary review labels [7,8]. Fit preprocessing and the model on an earlier reference period, preserve the feature schema and random seed, and save a reloadable artifact. Tune a small set of model settings and the review threshold on development scenarios, then freeze them before final testing.

Separate incompatible account populations. Use platform-specific models when sufficient records exist, or restrict a shared experiment to features with the same meaning and source coverage. Exclude raw account names, host identifiers and scenario labels from predictors. If the dataset cannot support a learned comparison, report that limitation and collect more baseline activity rather than presenting fixed rules as ML.

Behavior change and policy checks

Include differences between recent activity and the account's own historical profile in the model inputs. Compare the learned ranking with a simple statistical change score and with an inactivity/privilege rules baseline. Peer comparisons are optional when roles have enough comparable accounts. A new behavior pattern is a review signal; it is not proof of misuse or unnecessary access.

Run explicit policy checks alongside the model. A configured inactivity rule remains visible even if the anomaly model does not flag it. A 30-day inactivity condition requires adequate observation history or a trustworthy prior timestamp. Shorter lab thresholds or time-compressed replay must be marked as demonstration settings, with original event times retained.

Output state	Meaning	Required evidence
Review required	A policy condition or unusual behavior warrants a human check.	Trigger, observed values, baseline comparison and affected source.
No review recommended	No configured trigger was found in sufficiently covered evidence.	Coverage period and model version; this is not a guarantee that access is safe.
Insufficient evidence	Logs, identity mapping or history do not support a reliable recommendation.	Missing fields, unsupported source or required additional history.

Export account-level results in CSV and JSON with a sortable anomaly score, policy flags and evidence references. Present the score as a ranking, not as the probability of malicious access. Explain observed changes directly; claim feature attribution only if a suitable explanation method is actually implemented. Human reviewers remain responsible for retaining, investigating or removing access.

Evaluation and acceptance criteria

Controlled cases with real recorded events

Run documented scenarios with lab accounts and retain actual OS/application events. Keep scenario labels outside model inputs. Labels identify cases needing review, not confirmed misuse. Include legitimate variation as well as exceptions.

Scenario	Expected assessment	Important control
Stable normal activity	No review recommendation when evidence is adequate.	Vary ordinary hours and event counts across users.
Previously active account stops activity	Review when the configured inactivity condition is satisfied.	Confirm log coverage; distinguish absence from collector failure.
Activity category changes	Review an observable change from the account's earlier pattern.	Capture the actual audited activity; do not infer tasks from login events.
Role or privilege changes	Surface the inventory change and observed privilege use.	Include authorized changes as review-worthy but not automatically malicious.
New or sparsely observed account	Insufficient evidence where history is inadequate.	Do not hide these cases in the normal class.
Missing logs or ambiguous identity	Insufficient evidence with a data-quality reason.	Test source outage and duplicate usernames on different hosts.

Splits and fair comparisons

Fit on earlier data, select thresholds on development scenarios and preserve a later test period. Account for overlapping feature windows to prevent leakage. Add an account-group holdout where feasible and distinguish known-account monitoring from new-account behavior. Fit all preprocessing and reference statistics on training data only [9].

Compare rules, the statistical baseline, Isolation Forest and the hybrid on the same accounts and review budget. Count mandatory policy cases in the total workload. Report behavior-change performance separately to establish whether the model adds value beyond inactivity detection.

Measures and delivery gate

Report precision, recall, average precision [10], false alerts per 100 eligible assessments, performance at the agreed review capacity and unsupported-case counts. Measure runtime, memory, parsing coverage and repeatability. Lab results establish prototype feasibility; enterprise accuracy requires broader validation.

Acceptance requires Linux/Windows/CSV inputs, a fitted and reloadable model, all three output states, traceable evidence and reproducible baseline results. Measure detection performance alongside any workload reduction. Do not promise accuracy or time savings before testing.

Implementation plan and deliverables

The following six-week MVP plan assumes access to suitable lab machines and approximately 15-20 developer hours per week. It is an estimate, not a deadline stated by the stakeholder. Begin event collection in week 1; data adequacy determines when model evaluation can proceed.

Week	Work	Completion evidence
1	Confirm the review target and CSV schema. Configure Linux/Windows lab logging and begin collection.	Data dictionary, source examples and scenario plan.
2	Implement inventory/event parsers, identity joins and data-quality reporting.	Normalized dataset and tested parser fixtures.
3	Create account windows and change features. Fit the policy/statistical baselines and Isolation Forest.	Versioned features, fitted artifact and development results.
4	Freeze thresholds. Run held-out scenarios, source-failure tests and error analysis.	Evaluation report with sample counts and limitations.
5	Complete the command-line workflow and CSV/JSON evidence report.	Demonstration from raw inputs to ranked recommendations.
6	Fix defects, rerun the documented experiment and prepare the final demonstration.	Source, model, data description and reproducible handover.

Software and practical requirements

Use Python, pandas, scikit-learn and pytest, with locked environment versions. CSV/JSON files and SQLite are sufficient for the MVP. Use an authorized Windows installation and a Linux VM. Start the tabular model on CPU and check VM memory and dataset size locally; GPU training is optional.

Engineering and verification

Separate adapters, normalization, features, training, scoring and reporting. Version the data contract and model metadata. Test malformed timestamps, duplicates, mixed usernames, missing events, feature reproducibility, artifact reload and score direction. Keep credentials outside code and load trusted model artifacts only [11]. Score all eligible accounts, including those not flagged by rules.

Final deliverables and remaining decisions

Deliver the source, schemas, lab instructions, sample dataset, scenario manifest, model, feature configuration, evaluation report, review report and demonstration. Include a model card describing intended use, training coverage, failures and retraining conditions.

Confirm the deadline, lab resources, account count, permitted audit events and reviewer capacity. Limit detection claims to observable activity. Keep the existing AccessGuard dashboard optional until the model and evidence pipeline are complete.

Related work and references

ML-assisted access review already exists. Microsoft documents activity-based and organizational-affiliation recommendations [12]. This project's proposed contribution is a portable, testable prototype that combines local-account inventories with activity evidence and compares learned triage against simple baselines. It does not claim a new anomaly algorithm or a first-ever access-review system.

The expected result is an evidence-backed answer to a practical question: at the same review capacity, does learned behavior analysis identify additional review-worthy accounts without an unacceptable increase in false alerts? The lab will establish what can be demonstrated; an enterprise deployment would require broader data and operational validation.

[1] Project stakeholder discussion. Recording 2026-11-02 214117.mp3. User-supplied meeting recording and available transcript. Requirements source.

[2] systemd project. journalctl manual. https://www.freedesktop.org/software/systemd/man/latest/journalctl.html

[3] Microsoft Learn. Microsoft.PowerShell.LocalAccounts module. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.localaccounts

[4] Microsoft Learn. Event 4624: An account was successfully logged on. https://learn.microsoft.com/windows/security/threat-protection/auditing/event-4624

[5] Microsoft Learn. Audit Process Creation. https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/audit-process-creation

[6] Apache Software Foundation. mod_log_config and custom log formats. https://httpd.apache.org/docs/2.4/mod/mod_log_config.html

[7] Liu FT, Ting KM and Zhou ZH. 2008. Isolation Forest. IEEE ICDM, pp 413-422. https://cs.nju.edu.cn/zhouzh/zhouzh.files/publication/icdm08b.pdf

[8] scikit-learn documentation. IsolationForest. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

[9] scikit-learn documentation. Common pitfalls and recommended practices. https://scikit-learn.org/stable/common_pitfalls.html

[10] scikit-learn documentation. average_precision_score. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

[11] scikit-learn documentation. Model persistence. https://scikit-learn.org/stable/model_persistence.html

[12] Microsoft Learn. Review recommendations for Access reviews. https://learn.microsoft.com/en-us/entra/id-governance/review-recommendations-access-reviews

Technical sources checked on 3 November 2026. The proposed thresholds, sample size and schedule are planning assumptions; empirical results will be reported after implementation.